

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I B Sandhya(192311292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Sandhya

Annexure A

Scenario Based Assignment

5. **Scenario** : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario**: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

ANSWERS

Scenario-Based Assignment

1. Scenario

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic because roads and paths may suddenly become inaccessible, weather conditions can change, and movement costs vary depending on terrain

and weather. The drone has access to a heuristic estimate such as straight-line distance, partial real-time information about blocked paths, and variable movement costs.

The drone must quickly select routes while considering **minimum delay, route cost, and safety**. Two important search algorithms for this situation are **Greedy Best-First Search** and **A***.

i. Greedy Best-First Search in the Given Scenario

Greedy Best-First Search selects the node that appears closest to the destination according to the heuristic value.

The evaluation function is:

$$f(n) = h(n)$$

where **$h(n)$** represents the estimated distance from the current location to the destination.

For example, if the drone has three possible routes, it will prefer the route whose estimated remaining distance is smallest, even if that route has difficult terrain or bad weather.

Strengths

- It is generally **fast** because it focuses directly on the destination.
- It can be useful when the drone needs to make a **quick decision**.
- It requires relatively less search compared with algorithms that consider the complete path cost.
- A good heuristic can help the drone reach the destination quickly.

Weaknesses

- It does **not consider the cost already spent** reaching the current location.
- It may select a route that looks short but has high weather or terrain costs.
- It is **not guaranteed to find the optimal route**.
- Under uncertainty, an apparently short route may suddenly become blocked.
- Poor heuristic information can cause the drone to make unsafe or inefficient choices.

Therefore, Greedy Best-First Search is attractive when speed is the highest priority, but it may sacrifice route quality and safety.

ii. A* Search in the Given Scenario

A* combines the actual cost of reaching a location with the estimated cost of reaching the destination.

Its evaluation function is:

$$f(n) = g(n) + h(n)$$

where:

- **$g(n)$** = actual cost from the starting point to the current node.
- **$h(n)$** = estimated cost from the current node to the destination.
- **$f(n)$** = estimated total cost of the complete route.

For example, suppose one route is geographically short but has heavy flooding and another route is slightly longer but has safer and faster terrain. A* can consider both the distance and the accumulated travel cost.

How A* balances cost and heuristic

The **$g(n)$** value prevents A* from choosing a route simply because it appears close to the destination. The **$h(n)$** value helps it avoid unnecessarily exploring routes that are obviously far away.

Therefore, A* provides a balance between:

Actual route cost + Estimated remaining cost = Total estimated route cost

With an appropriate and admissible heuristic, A* can provide an optimal solution in a static environment.

However, because the environment is dynamic, the drone may need to **re-plan** when new information about blocked paths or weather becomes available.

iii. Impact of Heuristic Accuracy

The accuracy of the heuristic has a major impact on both algorithms.

Greedy Best-First Search

Greedy Best-First Search depends heavily on **$h(n)$** because it uses only the heuristic.

- **Accurate heuristic:** It can quickly select routes that lead toward the destination.
- **Inaccurate heuristic:** It may choose a bad route because it believes that route is closer to the destination.
- **Changing environment:** A previously accurate distance estimate may become misleading when flooding or weather changes.

Therefore, Greedy Best-First Search is highly sensitive to heuristic errors.

A*

A* uses both **$g(n)$** and **$h(n)$** .

- If the heuristic is accurate, A* can search efficiently.
- If the heuristic underestimates the actual remaining cost but remains admissible, A* can still provide an optimal solution in a static environment.
- If the heuristic is very poor, A* may explore many unnecessary nodes and become slower.
- If the heuristic overestimates costs, optimality is no longer guaranteed in the standard formulation.

Thus, A* is generally more robust than Greedy Best-First Search because it also considers the actual cost already incurred.

iv. Effect of Dynamic Changes and Blocked Paths

Dynamic changes create a major challenge for both algorithms.

Suppose the drone selects a path and suddenly receives an update that a bridge or road on that path is flooded.

Effect on Greedy Best-First Search

Greedy Best-First Search may quickly select another route based on the updated heuristic. This makes it useful for rapid re-routing.

However, repeated changes can cause the drone to repeatedly select routes that look promising but later become blocked. This can result in inefficient movement or repeated searching.

Effect on A*

A* can also respond to blocked paths by updating the search graph and calculating another route.

Its advantage is that it considers both **previous travel cost and future estimated cost**, so the new route can be selected more systematically.

However, standard A* is designed primarily for static search spaces. If the environment changes frequently, repeatedly running A* from the beginning can be computationally expensive.

For highly dynamic environments, **incremental or dynamic path-planning algorithms** such as D* Lite can be more suitable because they are specifically designed to efficiently update paths when the environment changes.

v. Recommended Algorithm

For this emergency medicine delivery scenario, A* would be the better choice between Greedy Best-First Search and A*.

Reasons

1. **Balances speed and route cost:**
A* considers both the actual cost and estimated remaining cost using $f(n) = g(n) + h(n)$.
2. **Better route quality:**
It is less likely than Greedy Best-First Search to choose a route simply because it looks geographically close.
3. **Considers terrain and weather:**
Variable movement costs can be incorporated into $g(n)$, allowing the algorithm to prefer safer and less costly routes.
4. **Better decision-making under uncertainty:**
When updated information is received, the search can be re-planned using the latest information.
5. **Optimality:**
With a suitable admissible heuristic and a static environment, A* can guarantee an optimal route.
6. **Safety:**
The cost function can assign high costs to dangerous areas, flooded regions, or severe weather conditions, helping the drone avoid risky routes.

Final Recommendation

A* is the most suitable choice when the drone must balance **real-time response, route efficiency, cost, and safety**. Greedy Best-First Search may be faster in some cases, but it can make poor decisions because it considers only the heuristic distance.

However, because the flood-affected environment is highly dynamic, the best practical solution would be to use **A* with real-time re-planning**, or a dynamic path-planning algorithm such as *D Lite** for frequent environmental changes.

Therefore:

Greedy Best-First Search → Faster but less reliable and not necessarily optimal.

A* → Better balance between speed, cost, and optimality.

Dynamic A/D Lite → More suitable when paths change frequently in real time.**

For an emergency medicine drone, **safety and reliable re-routing should take priority over selecting the absolute shortest-looking path.**

2. Scenario: AI-Based Traffic Signal Optimization in a Smart City

A smart city uses an AI system to optimize traffic signal timings across hundreds of intersections. The main objectives are to reduce **total waiting time, fuel consumption, and traffic congestion**. Since traffic flow changes continuously, the system must frequently adjust signal timings. However, the number of possible signal configurations is extremely large, making the problem difficult to solve using traditional search methods.

Formulation as an Optimization Problem

The problem can be formulated as a **combinatorial optimization problem**.

State

A state represents the signal configuration of all intersections, including:

- Green-light duration
- Red-light duration
- Yellow-light duration
- Signal phase sequence
- Coordination between neighboring intersections

Objective Function

The system tries to minimize a combined cost:

$$\text{Cost} = w_1(\text{Total Waiting Time}) + w_2(\text{Fuel Consumption}) + w_3(\text{Traffic Congestion})$$

where w_1 , w_2 , and w_3 represent the importance assigned to each objective.

The goal is:

Minimize Cost

The AI continuously evaluates traffic conditions and searches for better signal configurations.

Why Traditional Search is Inefficient

Traditional exhaustive search would try many possible signal combinations and compare their performance.

For hundreds of intersections, the number of possible configurations becomes extremely large. This causes:

- **Huge search space**

- High computational time
- Large memory requirements
- Difficulty responding to real-time traffic changes
- Repeated evaluation of many poor solutions

Therefore, instead of searching every possible solution, **local and stochastic optimization techniques** are more appropriate.

i. Hill Climbing

Hill Climbing is a local search algorithm that starts with an initial solution and repeatedly moves to a neighboring solution that improves the objective value.

For example, the AI may initially assign:

- 40 seconds green at Intersection A
- 50 seconds at Intersection B
- 35 seconds at Intersection C

It then changes one or more signal timings slightly and measures the new traffic performance.

If the new configuration reduces total waiting time, fuel consumption, or congestion, the system moves to that configuration.

The process continues until no neighboring configuration provides an improvement.

Advantages

- Simple to implement
- Requires relatively little memory
- Fast compared with exhaustive search
- Suitable for large optimization problems
- Can quickly improve an existing traffic-signal configuration

Limitations

The major problem is that Hill Climbing makes decisions based mainly on the current neighborhood. It may therefore stop at a solution that is good locally but not globally optimal.

This is particularly problematic in a smart-city environment where traffic conditions continuously change.

ii. Local Maxima, Plateaus, and Ridges

Although the traffic problem is formulated as a **minimization** problem, the same concepts can be explained using the equivalent idea of maximizing a fitness score.

1. Local Maximum

A local maximum is a solution that is better than all nearby solutions but worse than another solution somewhere else in the search space.

Real-world interpretation:

Suppose the AI changes signal timings at five nearby intersections and finds a configuration that significantly reduces congestion in that area. However, another configuration involving coordinated timings across the entire city could reduce congestion much more.

Hill Climbing may stop at the first good configuration because every small change around it appears worse.

2. Plateau

A plateau occurs when many neighboring solutions have almost the same evaluation value.

Real-world interpretation:

Changing a traffic signal from 40 to 41 or 42 seconds may produce almost no noticeable change in traffic flow. The algorithm therefore sees several configurations with nearly identical performance.

As a result, Hill Climbing may not know which direction to take and can spend time moving around the flat region or stop prematurely.

3. Ridge

A ridge is a narrow region where improvement requires a combination of changes rather than one simple change.

Real-world interpretation:

Changing the signal at only one intersection may increase congestion. But changing signals at three connected intersections together may significantly improve traffic flow.

Hill Climbing may reject each individual change because it looks worse temporarily, even though the combination would produce a much better solution.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing

Simulated Annealing improves on Hill Climbing by sometimes accepting a worse solution.

At the beginning, the algorithm has a high **temperature**, allowing it to accept some worse solutions. This helps it escape local optima.

As the temperature gradually decreases, the algorithm becomes more selective and increasingly behaves like Hill Climbing.

Example

Suppose the current traffic configuration has a cost of 1,000 and a new configuration has a cost of 1,020.

Hill Climbing would reject the new configuration immediately.

Simulated Annealing may temporarily accept it because that configuration could lead to another configuration with a much lower cost later.

This allows the algorithm to escape:

- Local optima
- Plateaus
- Some difficult search regions

Genetic Algorithm

A Genetic Algorithm uses a population of possible traffic-signal configurations rather than a single solution.

Each solution can be represented as a **chromosome**, where genes represent signal timings.

The main operations are:

1. **Selection** – Choose better traffic configurations.
2. **Crossover** – Combine parts of two good solutions.
3. **Mutation** – Randomly change some signal timings.
4. **Fitness evaluation** – Measure waiting time, fuel consumption, and congestion.
5. **Repeat** – Continue producing improved generations.

For example, one solution may perform well in the northern part of the city while another performs well in the southern part. Crossover can combine their useful characteristics into a new solution.

Genetic Algorithms are therefore less likely to become permanently trapped in a single local optimum.

iv. Recommended Approach

For this smart-city scenario, **Genetic Algorithm (GA)** is the most suitable primary approach when the goal is to optimize signal timings across hundreds of intersections.

Reasons for Choosing Genetic Algorithm

1. Scalability

The search space is extremely large. GA evaluates a population of candidate solutions instead of exhaustively checking every possible signal configuration.

2. Avoids Local Optima

Selection, crossover, and mutation provide exploration of different regions of the search space. This makes GA less vulnerable to local maxima, plateaus, and ridges than basic Hill Climbing.

3. Handles Multiple Objectives

The system has three objectives:

- Minimize waiting time
- Minimize fuel consumption
- Minimize congestion

These can be combined into a fitness function, or a multi-objective GA can be used to find trade-offs between them.

4. Adaptability

When traffic patterns change, the population can be updated and evolved toward configurations suitable for the new traffic conditions.

5. Suitable for Complex Relationships

Traffic at one intersection affects neighboring intersections. GA can optimize combinations of signal timings rather than making isolated decisions.

Comparison

Approach	Speed	Avoids Local Optima	Scalability	Adaptability
Hill Climbing	Very High	Poor	Good	Moderate
Simulated Annealing	Moderate	Good	Good	Good
Genetic Algorithm	Moderate	Very Good	Very Good	Very Good

Conclusion

Hill Climbing is simple and fast, but it can become trapped in local optima, plateaus, and ridges. Simulated Annealing improves exploration by occasionally accepting worse solutions, while Genetic Algorithms explore multiple solutions simultaneously using selection, crossover, and mutation.

For a **large, dynamic smart-city traffic network**, a **Genetic Algorithm is the recommended approach** because it provides good scalability, handles complex optimization objectives, and is less likely to get trapped in local solutions.

In a real implementation, a **hybrid approach** could be even better: use a Genetic Algorithm for global optimization and a fast local-search method for fine-tuning signal timings in real time. This would provide a good balance between **solution quality, scalability, and real-time adaptability**.

3. Scenario: Autonomous Rover on Mars

An autonomous rover is deployed on Mars to explore unknown terrain and collect scientific samples. It must safely navigate through unfamiliar areas while avoiding hazards such as craters and rocks. Since the rover has limited sensing capability and does not initially have a complete map, it must make decisions using the information available at each moment.

As the rover moves, it discovers new terrain and updates its knowledge. Therefore, this is a **real-time decision-making problem** that is well suited to an **online search agent**.

i. Why an Online Search Agent is Required

An **offline search algorithm** assumes that the complete environment, including the initial state, possible actions, and paths to the goal, is known before the search begins.

This assumption does not hold for the Mars rover.

The rover initially does not know:

- The complete terrain map
- The exact locations of rocks and craters
- Which paths are safe
- Whether a particular route is passable
- What obstacles may be discovered later

Therefore, the rover cannot create a complete route before starting its journey.

An **online search agent** performs searching and acting simultaneously. It:

1. Observes the current environment.
2. Selects an action.
3. Moves to the next location.
4. Receives new information.
5. Updates its knowledge.
6. Searches for the next suitable action.

For example, if the rover is moving toward a sample location and its sensors suddenly detect a crater, it can avoid that route and search for another safe path.

Thus, online search is appropriate because the rover must **act while discovering the environment**.

ii. Challenges of Partial Observability and Dynamic Environments

Partial Observability

The rover has limited sensors and therefore cannot observe the entire environment.

For example, it may only detect obstacles within a certain distance. A large rock behind another obstacle may remain unknown until the rover moves closer.

This creates several challenges:

- The rover does not have complete information.
- Some hazards may remain hidden.
- Sensor readings may contain uncertainty or noise.
- The best route may not be known initially.
- The rover must make decisions using incomplete information.

The rover therefore maintains a **belief or internal representation** of what it currently knows about the environment.

Dynamic Environment

Although Mars itself changes relatively slowly, the rover's operational environment can change from its perspective.

Examples include:

- Dust storms reducing visibility
- Changing terrain conditions
- Rover wheel slippage
- Newly detected obstacles
- Communication delays
- Unexpected terrain conditions

A route that was considered safe earlier may become unsuitable after new information is received.

Therefore, the rover must continuously **re-evaluate and update its route**.

iii. How the Rover Builds and Updates Its Internal Model

The rover gradually creates an internal model of its surroundings using sensor observations and previous actions.

Step 1: Initial Knowledge

The rover begins with limited information, such as:

- Starting position
- Approximate destination
- Known terrain information
- Sensor capabilities

Step 2: Sense the Environment

The rover uses sensors such as cameras, range sensors, and other onboard instruments to identify:

- Rocks
- Craters
- Slopes
- Safe terrain
- Potential sample locations

Step 3: Update the Map

The newly observed information is added to the rover's internal map.

For example:

Unknown area → Sensor observation → Identified obstacle → Updated map

Step 4: Update Route

After updating its model, the rover checks whether its current route is still safe.

If the route contains a newly discovered crater, the rover searches for an alternative route.

Step 5: Repeat

This process continues throughout the mission:

Sense → Update Model → Search → Act → Sense Again

This allows the rover to gradually improve its knowledge while exploring.

iv. Comparison with Uninformed and Informed Search

Uninformed Search

Uninformed algorithms such as **Breadth-First Search (BFS)** and **Depth-First Search (DFS)** do not use additional knowledge about how close a state is to the goal.

For a Mars rover, these approaches have limitations because:

- They may explore many unnecessary locations.
- They do not naturally prioritize safer or more promising terrain.
- BFS can require significant memory.
- DFS may follow a dangerous or unproductive path for too long.
- They are less suitable when the environment is unknown and continuously changing.

Informed Search

Informed algorithms such as **A*** use additional information, called a heuristic, to guide the search.

For example, the rover can use estimated distance to the sample location as a heuristic.

A* considers:

$$f(n) = g(n) + h(n)$$

where:

- **$g(n)$** = cost already incurred
- **$h(n)$** = estimated remaining cost

For the rover, the cost can include not only distance but also:

- Terrain difficulty
- Energy consumption
- Risk of obstacles
- Slope
- Travel time

Therefore, informed search can make better decisions than uninformed search when useful information is available.

However, standard A* still assumes that the search space is sufficiently known. In a continuously unknown environment, it may need to be repeatedly re-planned as new information becomes available.

v. Most Suitable Approach

For the Mars rover, an **online informed search approach with continuous re-planning** is the most suitable.

The rover should combine online decision-making with heuristic information whenever reliable information is available.

Why This Approach Is Suitable

1. Handles uncertainty

The rover does not need a complete map before beginning its mission. It can make decisions using currently available information.

2. Adapts to new information

When sensors discover a crater or rock, the rover can update its internal model and select a different route.

3. Improves safety

The search cost can include risk. For example, dangerous terrain can be assigned a very high cost so that the rover prefers safer routes even if they are slightly longer.

4. Saves resources

Mars rovers have limited energy and computational resources. A heuristic can reduce unnecessary exploration and help the rover reach useful locations more efficiently.

5. Supports real-time decisions

The rover can repeatedly follow the cycle:

Observe → Update → Search → Act → Observe

This makes it suitable for autonomous exploration.

Final Recommendation

The Mars exploration problem is best handled using an **online informed search agent** rather than a purely offline or uninformed search algorithm.

An approach based on **online A*** or other incremental/re-planning search techniques can use heuristic information while continuously updating the route based on new observations. For highly uncertain environments, the system can also incorporate risk and energy costs into its evaluation.

Therefore:

Offline Search → Not suitable because the complete map is unavailable.

Uninformed Search → Simple but inefficient and does not use available terrain information.

Informed Search → More efficient because it uses heuristic knowledge.

Online Informed Search → Most suitable because it combines adaptability, heuristic guidance, uncertainty handling, and safety.

For a Mars rover, **safe and adaptive decision-making is more important than simply finding the shortest path**, because an apparently short route may contain unknown hazards or consume excessive energy.

4. Scenario: AI-Based University Exam Timetabling

A university wants to design an AI system that automatically generates an exam timetable. The timetable must satisfy several constraints, including preventing student exam clashes, managing a limited number of halls, respecting subject ordering requirements, considering faculty availability, and providing special accommodations for students.

This problem can be naturally modeled as a **Constraint Satisfaction Problem (CSP)**.

Formulation as a Constraint Satisfaction Problem

A CSP consists of three main components:

CSP = Variables + Domains + Constraints

For the exam timetable, the objective is to assign a valid time and hall to every examination while satisfying all required constraints.

i. Variables, Domains, and Constraints

1. Variables

Each exam can be represented as a variable.

For example:

- E1 = Mathematics exam
- E2 = Operating Systems exam
- E3 = Database Management Systems exam
- E4 = Computer Networks exam

If there are 100 exams, the CSP will contain 100 exam variables.

Each variable represents the **scheduled assignment** of an exam.

For example:

E1 = (Monday 10 AM, Hall 1)

2. Domains

The domain contains all possible values that can be assigned to each exam.

For example:

Time slots:

- Monday 9–11 AM
- Monday 2–4 PM
- Tuesday 9–11 AM
- Tuesday 2–4 PM

Halls:

- Hall 1
- Hall 2
- Hall 3

Therefore, an exam's domain could contain combinations such as:

(Monday 9 AM, Hall 1)

(Monday 9 AM, Hall 2)

(Tuesday 2 PM, Hall 3)

The system removes invalid combinations as it applies constraints.

3. Constraints

The constraints define which assignments are allowed.

Constraint 1: No Student Exam Overlap

If two exams share students, they cannot occur at the same time.

For example:

Students taking Mathematics $\cap$ Students taking Physics $\neq \emptyset$

Therefore:

Time(Math) $\neq$ Time(Physics)

Constraint 2: Limited Exam Halls

The number of exams scheduled at a particular time cannot exceed the number of available halls.

For example, if there are only 3 halls, at most 3 exams can be conducted simultaneously.

Constraint 3: Subject Ordering

Some subjects must be conducted before others.

For example:

Data Structures $\rightarrow$ Advanced Algorithms

Therefore:

Time(Data Structures) $<$ Time(Advanced Algorithms)

Constraint 4: Faculty Availability

An exam requiring a particular faculty member must be scheduled when that faculty member is available.

For example:

If Professor A is unavailable on Tuesday morning:

Time(Exam supervised by Professor A) $\neq$ Tuesday morning

Constraint 5: Special Accommodations

Students with special requirements may need specific time slots, accessible halls, extra time, or other arrangements.

For example:

Exam(Student with accommodation) = Accessible Hall

The system must ensure that these requirements are satisfied.

CSP Representation

Component	Example
Variables	Exams such as Mathematics, DBMS, OS

Component	Example
Domains	Available time slots and halls
Constraints	No clashes, hall capacity, ordering, faculty availability, accommodations
Solution	Complete timetable satisfying all constraints

ii. Backtracking Search

Backtracking Search assigns values to variables one at a time and checks whether the partial timetable violates any constraint.

The basic process is:

Start → Select Exam → Assign Time/Hall → Check Constraints → Continue or Backtrack

Example

Suppose the system schedules:

Mathematics → Monday 9 AM, Hall 1

It then attempts:

Physics → Monday 9 AM, Hall 2

If some students are enrolled in both Mathematics and Physics, this assignment violates the no-overlap constraint.

The algorithm therefore **backtracks** and tries another assignment:

Physics → Monday 2 PM, Hall 1

If this assignment satisfies the constraints, the algorithm continues with the next exam.

If the algorithm reaches a situation where no valid assignment exists, it goes back to an earlier decision and tries another value.

Advantages

- Systematically explores possible timetables.
- Guarantees a solution if one exists and the search is exhaustive.
- Can handle many different types of constraints.
- Easy to combine with constraint-solving techniques.

Limitation

Basic backtracking can become extremely slow as the number of exams, students, halls, and time slots increases because the number of possible combinations can grow exponentially.

Therefore, backtracking should be combined with **constraint propagation and intelligent variable/value selection**.

iii. Constraint Propagation and Forward Checking

Constraint propagation reduces the search space before the algorithm explores unnecessary assignments.

Forward Checking

After assigning a value to an exam, forward checking examines the remaining unassigned exams and removes values that would violate constraints.

Example

Suppose:

Mathematics = Monday 9 AM, Hall 1

If Physics shares students with Mathematics, Monday 9 AM is removed from Physics's domain.

Initially:

Physics = {Mon 9, Mon 2, Tue 9, Tue 2}

After forward checking:

Physics = {Mon 2, Tue 9, Tue 2}

The invalid value is removed immediately.

This prevents the system from exploring an invalid timetable later.

Benefits

- Reduces the search space.
- Detects conflicts earlier.
- Reduces unnecessary backtracking.
- Improves computational efficiency.
- Works especially well when many constraints interact.

Other Useful CSP Techniques

The system can also use:

Minimum Remaining Values (MRV):

Choose the exam with the fewest remaining possible assignments first.

Degree Heuristic:

Choose the exam involved in the largest number of constraints.

Least Constraining Value (LCV):

Choose the assignment that leaves the most options available for other exams.

Combining these techniques can significantly improve performance.

iv. Real-World Challenges and Best Strategy for Scalability

Real-World Challenges

A real university timetable is more complicated than a simple CSP.

1. Large Number of Courses

As the number of courses increases, the number of possible schedules increases dramatically.

2. Large Student Population

Thousands of students may create a large number of exam-conflict relationships.

3. Limited Resources

The university may have a limited number of:

- Exam halls
- Faculty members
- Accessible rooms
- Computers for online examinations

4. Faculty Availability

Faculty schedules may change because of meetings, leave, or other university activities.

5. Special Requirements

Some students may require:

- Accessible rooms
- Additional time
- Specific seating arrangements
- Separate examination facilities

6. Changes at the Last Minute

An exam may need to be rescheduled because of faculty unavailability, hall problems, or unexpected university events.

Therefore, the system should be capable of modifying part of the timetable without rebuilding the entire schedule.

Recommended Strategy

For a large university, the best strategy is a **hybrid CSP approach using backtracking, constraint propagation, and intelligent heuristics**.

The recommended process is:

CSP Modeling

↓

Constraint Propagation / Forward Checking

↓

MRV + Degree Heuristic

↓

Backtracking Search

↓

Conflict Detection

↓

Re-planning when necessary

Why This Strategy Is Best

1. Scalability:

Constraint propagation removes invalid possibilities early, reducing the search space.

2. Faster Search:

MRV selects the most constrained exam first, helping detect conflicts early.

3. Better Resource Utilization:

The system can efficiently manage halls, faculty, and time slots.

4. Adaptability:

If a faculty member becomes unavailable or a hall becomes unavailable, the system can update the affected assignments and re-solve the relevant part.

5. Constraint Satisfaction:

The approach can simultaneously handle student conflicts, hall limitations, ordering constraints, faculty availability, and special accommodations.

Conclusion

University exam scheduling is an excellent example of a **Constraint Satisfaction Problem** because it involves variables, domains, and multiple constraints.

Basic backtracking can find a valid timetable, but it may become inefficient for large universities. **Forward checking and constraint propagation** improve performance by eliminating invalid assignments early.

Therefore, the most suitable approach is a **hybrid CSP solver using backtracking + forward checking + MRV + degree heuristics**. For very large universities, this can be further combined with optimization techniques or specialized CSP/constraint-programming solvers.

This approach provides a good balance between **correctness, scalability, efficiency, and adaptability**.

5. Strategic Game AI for Real-Time Decision Making

Minimax and Alpha-Beta Pruning

A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, make decisions within strict time limits, evaluate complex game states with many possible moves, and handle an extremely large decision tree.

Minimax and Alpha-Beta Pruning can be used to make intelligent decisions while reducing unnecessary computation.

i. How Minimax Models the Problem

Minimax is a decision-making algorithm used for adversarial games where two players have competing objectives.

The two players can be considered:

- **MAX player:** The AI, which tries to maximize its chance of winning.
- **MIN player:** The human opponent, who tries to minimize the AI's chance of winning.

The game is represented as a **game tree**.

Game Tree Structure

- **Root node:** Current game state.
- **Branches:** Possible actions or moves.
- **Child nodes:** Game states resulting from those moves.
- **MAX levels:** AI chooses the move with the highest value.
- **MIN levels:** Opponent chooses the move with the lowest value.
- **Leaf nodes:** Final or depth-limited game states.

For example:

Current State

→ AI Move A
→ AI Move B
→ AI Move C

For each AI move, the algorithm considers possible responses by the human player.

The AI assumes that the opponent will always choose the move that is most disadvantageous to the AI. It therefore chooses the move that gives the best result under the opponent's strongest response.

Basic Minimax Logic

At a **MAX node**:

Choose the maximum child value.

At a **MIN node**:

Choose the minimum child value.

This allows the AI to select a move while anticipating the opponent's response.

ii. Computational Challenges of Large Game Trees

The major problem with Minimax is the enormous size of the game tree.

If:

- b = average number of possible moves (branching factor)
- d = search depth

Then the approximate number of nodes explored is:

$$O(b^d)$$

For example, if a game has 10 possible moves at every state and the AI searches 6 levels deep:

$$10^6 = 1,000,000 \text{ nodes}$$

If the branching factor or depth increases, the number of states grows extremely quickly.

Major Challenges

1. Large Branching Factor

Real-time strategy games may have many possible actions, such as:

- Moving units
- Attacking enemies
- Building structures
- Collecting resources
- Researching technologies
- Defending locations

This creates a huge number of possible states.

2. Strict Time Limits

The AI cannot spend several seconds calculating one move while a human player continues playing.

3. Complex Game States

A state may contain information about:

- Hundreds of units
- Resources
- Buildings
- Locations
- Health
- Technology
- Enemy positions

Evaluating each state can itself be computationally expensive.

4. Limited Search Depth

Because the tree is huge, the AI usually cannot search until the actual end of the game.

Instead, it searches to a limited depth and uses an **evaluation function** to estimate how good the resulting state is.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot affect the final decision.

It produces the **same optimal Minimax result** as full Minimax but can examine far fewer nodes when moves are ordered effectively.

Alpha-Beta maintains two values:

- **Alpha (α):** The best value that MAX can guarantee so far.
- **Beta (β):** The best value that MIN can guarantee so far.

Detailed Reasoning

Suppose MAX has already found a move with value:

$$\alpha = 8$$

Now, while examining another branch, MIN finds a response with value:

$$\beta = 6$$

Since MIN can already force a value of 6 or lower in this branch, MAX would never choose this branch over the previously discovered option with value 8.

Therefore, the remaining unexplored nodes in that branch do not need to be evaluated.

The pruning condition is:

If $\alpha \geq \beta$, prune the remaining branches.

Example

Suppose MAX has two possible moves:

Move A

- MIN can force a value of 5.

Move B

- During evaluation, MAX already finds a possible value of 8.

- MIN then finds a response producing 6.

Since MAX already has an alternative worth 8, the branch producing 6 cannot become preferable to MAX.

The remaining possibilities under that branch can therefore be skipped.

Benefits

Alpha-Beta Pruning:

- Reduces the number of nodes evaluated.
- Allows deeper search within the same time limit.
- Produces the same result as Minimax when applied correctly.
- Makes game AI significantly faster.
- Is especially effective when good moves are examined first.

With ideal move ordering, the effective complexity can improve from approximately:

$O(b^d)$

to approximately:

$O(b^{d/2})$

This means the AI can potentially search much deeper without requiring proportional increases in computation.

iv. Evaluation Function

Because the AI cannot search the complete game tree, it needs an **evaluation function** to estimate the quality of non-terminal game states.

A possible evaluation function is:

Evaluation(State) = w_1 (Military Strength) + w_2 (Resource Advantage) + w_3 (Territory Control) + w_4 (Technology Advantage) + w_5 (Unit Health) + w_6 (Base Safety) – w_7 (Enemy Strength)

The weights determine the importance of each factor.

Important Factors

1. Military Strength

Measures the number and quality of friendly units compared with the opponent.

More effective military forces generally indicate a stronger position.

2. Resource Advantage

Considers resources such as:

- Gold
- Food
- Energy
- Minerals

A player with greater resources can build more units and structures.

3. Territory Control

Measures how much strategically valuable territory the AI controls.

Controlling important areas can provide resources and tactical advantages.

4. Technology Advantage

Higher technological development may provide:

- Stronger units
- Better weapons
- Improved defenses
- Special abilities

Therefore, technology level should contribute to the evaluation.

5. Unit Health

Healthy units are more valuable than damaged units because they can continue fighting effectively.

6. Base Safety

The AI should consider whether its main base is protected.

A strong army is not useful if the opponent can easily destroy the base.

7. Enemy Strength

The opponent's military strength, resources, territory, and technology should also be considered.

This prevents the AI from focusing only on its own advantages.

Why Multiple Factors Are Needed

A single factor such as the number of units is insufficient.

For example, the AI may have more units but very little resource supply and a poorly defended base. Another player may have fewer units but superior technology and territory.

A multi-factor evaluation function allows the AI to make more realistic decisions.

v. Recommended Strategy for Balancing Optimality and Real-Time Performance

For a real-time strategy game, using **full-depth Minimax is impractical** because the game tree is too large.

The recommended strategy is:

Depth-Limited Minimax + Alpha-Beta Pruning + Good Move Ordering + Evaluation Function + Time Limit

1. Depth-Limited Search

Instead of searching until the end of the game, search only to a fixed depth.

This keeps computation manageable.

2. Alpha-Beta Pruning

Remove branches that cannot influence the final decision.

This allows the AI to search deeper within the available time.

3. Good Move Ordering

Evaluate promising moves first.

For example:

- Winning attacks
- Defensive moves
- High-value resource captures
- Moves threatening the opponent's base

Good move ordering makes Alpha-Beta pruning much more effective.

4. Iterative Deepening

The AI can search at increasing depths:

Depth 1 → Depth 2 → Depth 3 → Depth 4 → ...

If the time limit is reached, the AI uses the best move found at the previous completed depth.

This is particularly useful in real-time games because the AI always has a valid move available.

5. Time-Limited Decision Making

The AI should monitor computation time and stop searching before exceeding the allowed decision time.

Recommended Architecture

Current Game State



Generate Possible Moves



Order Promising Moves First



Depth-Limited Minimax



Alpha-Beta Pruning



Evaluation Function



Iterative Deepening



Best Move Within Time Limit

Conclusion

Minimax provides a strong framework for modeling competitive decision-making because it considers both the AI's actions and the opponent's best responses. However, the enormous size of real-time strategy game trees makes complete Minimax computationally expensive.

Alpha-Beta Pruning significantly improves efficiency by eliminating branches that cannot affect the final decision while maintaining the same Minimax result.

The best practical strategy is therefore **time-limited, depth-limited Minimax with Alpha-Beta pruning, strong move ordering, a carefully designed evaluation function, and iterative deepening.**

This approach provides a good balance between **optimality, computational efficiency, adaptability, and real-time performance**, allowing the AI to make strong decisions without exceeding the game's time constraints.